GOAC User Manual: Global Optimization of Atomistic Configurations by Coulomb energies

Konstantin Köster, Tobias Binninger, and Payam Kaghazchi

Contents

1 General Usage

GOAC can be directly accessed as a command line tool and the following flags are available:

- -h, –help
  - Show this help message and exit.
- -f CIF_FILE, –file=CIF_FILE
  - Read cif from FILENAME.
  - Provide the name of a valid cif by `-f file.cif`. GOAC uses PYMATGEN[1] for cif handling which is usually quite robust. However, if you are struggling with a specific cif we recommend to open the cif in VESTA[2] and export from there a fresh cif after converting to a different file format. This trick usually resolves any issues in reading a cif by GOAC.
- -p PROPERTIES, –property=PROPERTIES
  - Add properties as 'Na*=1.0' for charges or 'Na1=fixed' to fix sites.
  - Properties can be added as `-p "Na*=1.0"` to provide oxidation states for all elements (in this case Na) when performing an energy calculation. It is important to note that for all species no charge is given for, charges of zero are assumed and the code will also proceed if the total structure is not charge-balanced. Therefore, it is important to check the correctness of all charges in the problem statement output at the beginning of the calculation. For structures with multiple sites of the same element, e.g., Na0, Na1, ..., properties of all sites can be changed by applying the wildcard "*" as in the aforementioned example or properties of a certain site can be set by `-p "Na0=1.0"`. Next to providing charge states also `-p "Na*=fixed"` might be used to fix sites in the optimization problem. Next to the wildcard this option can be also applied to specific sites. If this option is selected the optimized structure will contain partial occupations for the corresponding sites. If one species in a multi-species site is fixed, all species on that site are fixed as well. For energy calculations the charges of the fixed species are averaged over all site positions. To check that the properties were passed correctly to GOAC, the problem summary that is outputted in the beginning of the calculation should be carefully checked in terms of charge states and sites that are iterated.

- -d, –dry
  - Set to perform dry run.
  - Setting this flag can be useful to verify that all properties are set correct and to get some general information about the problem such as number of configurations, number of atoms, and the composition and charge of the final structure. The written composition should be checked to ensure that the partial occupations are matched as desired as they will be determined by mathematical rounding from the occupation in the cif to the closest integer number that is accessible in the supercell.
- -y SOLVER_OPTIONS, –solver_options=SOLVER_OPTIONS
  - File with parameters for the set solver.
  - If not provided, default settings will be used but it can be very beneficial/necessary to adjust these options. More details on the available options are described in chapter 3 and chapter 4 of this manual for the different optimization strategies.
- -s SOLVER, –solver=SOLVER
  - Name of solver to solve given Coulomb problem.
  - Currently available options are `Gurobi` and `Gurobi-Heuristic` for usage of Gurobi [3]. For the internal Fortran heuristics one can choose from: `Greedy`, `Random`,`Random-BB`, `Random-LM`, `Random-MC`, `Random-SA`, `Random-REMC`, `Random-GA`, `Random-REGA` and `Random-Hybrid`. The details of these solver are explained in the corresponding chapters (chapter 3 and chapter 4) of this manual and technical details are discussed in the publication associated with GOAC. It should be emphasized that it is also possible for advanced users to create new, hybrid methods of the existing approaches in Python.
- -n N, –number=N
  - The 'n' best solutions are returned.
  - For internal heuristics, this parameter determines how many independent runs are started in parallel and details are discussed in chapter 4.
  - Depending on the solver, solutions are not checked for uniqueness and large n might be required to obtain different structures. Even if structures are checked for uniqueness, GOAC does not account for symmetries and therefore the best structures might be symmetry-equivalents and larger n are required to obtain the second, third, etc. best structures.
- -o OUT_NAME, –out-name=OUT_NAME
  - The out-name is written in front of all output-files.

- -w WRITE, –write=WRITE
  - Set number of cif files that will be written.
  - Energies of all *n* structures (and even more if samples is specified in the option file, cf. chapter 4) will be listed. However, only for the *w* lowest structures cif will be written if this flag is specified.
- -a, –write_energy
  - Set to write files with energy data.
  - The files "const", "alpha", and "beta" will be written that contain all the components of the energy matrices. In combination with the read flag this option can be useful to optimize heuristic parameters for a given problem or to try different heuristics without performing an energy calculation every time.
- -b, –write_part_struct
  - Set to write all structures to calculate energy data.
  - If set, cif for all components of the energy matrices are written. Thus, adding this flag will create a huge number of files. However, writing these files allows advanced users to use different energy evaluation methods than Coulomb within a second order expansion.
- -r, –read_energy
  - If set, read energies from files. Files must exist by running a calculation with "-a" first.
  - If this setting is applied, all provided charge states will be ignored and thus do not need to be specified. No check of the provided values is performed such that using the energies of a larger problem for a smaller one will not result in an error as enough energy values are provided. Users should be careful to always use the exact same problem statement in terms of material (cif), assumed charge states, supercell size, and fixed sites as otherwise the code will either fail and/or incorrect energies are obtained.
- -e SUPERCELL, –supercell=SUPERCELL
  - Dimensions of supercell given as "axbxc" with a,b,c being integers.
  - Internal creation of a supercell of the provided cif. By default `-e "1x1x1"` is employed but each "1" can be replaced by any positive integer number.
- test
- -x, –scan_supercells
  - If this flag is set, GOAC runs an optimization for all supercell-size combinations up to the supercell specified by `-e "axbxc"`.

- The process can be computationally demanding but can yield lower energy structures as smaller cells are easier to optimize and gives indications which supercell-size is required to represent the lowest energy configuration. Energies per unit cell of all optimizations are written to "Supercell-Scan-Report.txt".

The flags can be called from the command line as also shown in the example in the README:

```
python GOAC/ -f LCO.cif -p "Li*=1.0" -p "Co*=3.5" -p "O*=-2.0" -s
"random-mc" -n 4 -w 1
```

Moreover, experienced users can write their own Python scripts by utilizing the classes and methods provided by GOAC. To do so, it is recommended to familiarize with the __main__.py and the different solver classes. Some more details are discussed in the next chapter 2.

Beyond this manual, it it probably helpful to have a detailed look at the calculations in the example folder to understand how to use the functionalities of GOAC.

2 Energy Calculation

When using GOAC to prepare the energy matrices by Coulomb (Ewald) energies two main points should be considered:

1. It must be checked that the desired charge states are interpreted correctly as non-existing given properties will be simply ignored. Therefore, the summary in the beginning of program output, which also lists all considered charge states for each site species and the total charge of the final structure, should be carefully checked for an energy calculation run. GOAC will also proceed with a overall charged structure as this might be desired for certain problems. Species with no charge state are automatically assumed to be 0 (not considered in energy calculation).
2. Parallelization of energy calculations is realized via openMP and can be controlled by the environment variable "OMP_NUM_THREADS".

The energy calculation might be entirely skipped by providing files containing the necessary energy components along with the "-r" flag. In that case no sanity check is performed so the user should ensure that energy files that correspond to the exact problem (cif, supercell size, fixed sites) that should be solved are provided. To pre-calculate energy files, a run with "-s 'random'" and "-n 1" along with setting "-a" might be performed as with these setting no time will be spend on the optimization and GOAC finishes almost immediately after the energy calculation.

More advanced users might provide the energy files manually and calculate the energy components by different methods. All corresponding structure files can be outputted by setting the "-b" flag. For the constant energy part the file "const" should simply contain the energy of the "const.cif" file. For the first order terms the self energy and alpha energies should be calculated from the corresponding cif that have "-" separated the i and j index of the structure, e.g. "alpha-1-2.cif". The sum of the energies should be written to the "alpha" files by `for i {for j}`. The same holds true for the second order term where energies must be calculated from the "beta-i-j-k-l.cif" files and be stored in the "beta" file by `for i {for j {for k {for l}}}`. Care must be taken as GOAC assumes that the energies in the "beta" file are already corrected for self interactions.

3 Exact Optimizations

3.1 Creating MPS model files

As soon as any kind of Gurobi solver is selected, an MPS file will be written which might be used with any other optimization software that is capable of processing MPS files. By giving options explicitly (see next section for details), a very short run time might be selected to just output the MPS file without attempting to solving it.

3.2 Using Gurobi for Optimization

Both, the `Gurobi` and `Gurobi-Heuristic` solver have a minimal parameter set in GOAC that will be used by default. To enable parallelization, the "Threads" parameter should be specified in the options files passed to GOAC. However, if an file with options is passed via the "-y" flag the solver options will be read exclusively from the options file. All parameters that can be tuned in the Gurobi solver software (Parameters form Python API) can be wrapped via the option file by giving single space-separated parameter-value pairs per line. Once the MPS file is created, it is of course also not required to use GOAC at all and the model might be processed directly with Gurobi. For details on how to use Gurobi we refer to their official documentation: https://www.gurobi.com/documentation/.

4 Heuristic Optimizations

All options can be passed to GOAC as a file by the "-y" flag. The file should contain the exact parameter names as listed here and the desired value(s) separated by space. However, if no options file is provided, default parameter sets for all algorithms exist. However, it is highly recommended (and also necessary) to tune the available options for a specific problem. Moreover, setting the environmental variable "PYTHONUNBUFFERED" to True is necessary to obtain all outputs in the correct order.

4.1 Random

A Fortran routine that randomly selects a site-species and position and then places the corresponding ion if it is allowed by the occupation constraints of the atomistic combinatorial problem. The routine continues this procedure iteratively until a complete valid solution is achieved (e.g, all occupancy constraints are fully matched).

The "-n" flag that can be passed via the command line determines how many random structures will be created. The solver option is "-s 'random'".

4.2 Greedy

The greedy heuristic is a common inexpensive benchmark that can be the upper bond when comparing other methods or which can be used as a starting point for more advanced optimization techniques. Sites with partial occupations are filled iteratively while always the species and position with the lowest energy out of all possibilities is selected to place an ion. The Fortran implementation in GOAC also supports nested selection where the algorithm continues not just with the lowest but the n-lowest structures in each iteration. To avoid combinatorial explosion, the pool size is always reduced to n structures after two steps such that at most n^2 structures are considered per step.

A standard greedy algorithm is performed which naturally leads to just a single solution and can be called by "-s 'greedy'". However, if it is called with the "samples" option in the option file (passed y the "-y" flag) is greater than 1, each greedy step will continue with the $samples$ lowest energy solutions while the maximum amount of considered structures is limited to $samples \times samples$ per greedy step to avoid a combinatorial explosion. It is also important to highlight that choosing a

higher $samples$ might not find low energy structures that were obtained at lower $samples$ as there might be some intermediate greedy steps where lower energy structures are available resulting in different paths for the final solutions. Moreover, scaling with larger $samples$ is expensive so it is usually most useful to run the greedy solver with "samples 1" in the options file.

4.3 Gradient Descent

This Fortran routine performs a local minimization for a given (usually random) starting solution. For a starting solution all neighbouring solutions that can be accessed by swapping one iterative ion with an vacancy or two iterative ions with each other are evaluated and the lowest energy structure is selected until no direct neighbour structure with lower energy exists. This method can therefore be considered as a Gradient Descent for the discrete atomistic combinatorial problem and guarantees to find a local minimum. The routine in GOAC also supports to continue with the n lowest energy neighbours in each iteration while the solution pool is always reduced to n solutions after two steps such that at most n^2 solutions are considered per step.

The optimizer can be accessed by choosing the "random-lm" solver (LM for local minimizer) with help of the "-s" flag. The "-n" flag determines here how many local minimizations will be performed. However, n also determines how many optimizations are performed in parallel, in agreement with the environment variable "OMP_NUM_THREADS". Performance strongly depends on problem size and one should check how many samples can finish within the given computational resources. Alternatively, the algorithm also allows to (roughly) stop by time ("stop_time") or by optimizations without improvement of the global minimum ("stop_steps_no_improve"). To use these options, the parameter names should be written in one line each of the options file followed by their desired value separated by space while the stop time is in the unit of seconds. Negative values disable these settings. To use the abortion criteria of steps with no improvement the "tol" parameter should be specified in the options file as well to avoid resetting the steps with no improvement counter due to numerical fluctuations. An options file can be passed to GOAC with help of the "-y" flag. For complex problems it can take some time before the first solutions are written as only local minima after an successful local optimization are outputted. Moreover, the samples ("samples") parameter can be specified in the options file to store the lowest energy samples for each performed local optimization and to continue on lowest "samples" optimization paths per minimization step. The output of the final n lowest energy structures as defined by the "-n" flag chooses then from the whole solution pool of "samples"$\times n$ solutions. The "tol" parameter in the options file will also be used to only output structures from the pool that are, within "tol", unique in energy. This ensures to output symmetry-equivalent structures only once but might also filter non-symmetry-equivalent structures for very specific problems. If you fear your problem suffer from this behaviour, setting "tol" to a negative value in the option file will completely disable this filtering.

4.4 Monte Carlo

GOAC contains a Fortran implementation of an Metropolis Monte Carlo simulation[4] for the atomistic combinatorial problem. Again, a random valid starting solution is generated and a random neighbouring solution is chosen. A neighbouring structure is a structure that can be accessed by a single atom-swap. Then, the Metropolis criteria ($P = \min[1, \exp(-\Delta E/kT)]$, with ΔE being the energy difference of the current structure and the selected neighbour, k the Boltzmann constant, and T the simulation temperature) is used to determine the probability at which the random selected neighbouring structure is accepted. The new structure is then accepted or rejected by the calculated probability and the whole procedure is repeated for a desired amount of simulation steps.

For the Monte Carlo solver the "random-mc" solver must be selected. Similar to the gradient descent algorithm, the "-n" flag controls how many MC runs for independent starting structures are performed. Again, also up to the value of the environment variable "OMP_NUM_THREADS" independent Monte Carlo runs are performed at the same time. For Monte Carlo, the "samples" option in the option file, that is passed via the "-y" flag, determines how many lowest energy structures are stored per independent Monte Carlo run. The output of the final n lowest energy structures as defined by the "-n" flag chooses then from the whole solution pool of "samples"$\times n$ solutions. Next to the run time limitation in seconds ("stop_time") and optimization steps (Monte Carlo steps, therefore usually high values are required for Monte Carlo for this setting) without improvement of the global minimum ("stop_steps_no_improve") options to abort a run. Abortion of steps without improvement and selection of just unique structures in the solution pool also requires the "tol" setting in the options file to specify at which tolerance in energy structures should be considered to be different (real difference and no numerical fluctuations). Monte Carlo requires a few additional options, these options are "mc_steps", an integer that describes the amount of Monte Carlo steps to be performed, "mc_kt" which defines the simulation temperature in eV (energy, Boltzmann's constant can be used to transform to temperatures and *viceversa*), and "mc_write_steps" which defines after how many Monte Carlo steps an output is written (Monte Carlo can be quite verbose so it is usually best to set this parameter to a high value).

4.5 Simulated Annealing

This routine builds on the aforementioned MC routine but in addition, the temperature can be slowly reduced to 0 K during the simulation run. With a high starting temperature and a sufficiently slow cooling rate the chance of finding minimum energy structures can be increased.

It can be called by "random-mc" as well but only if "random-sa" is selected by the "-s" flag, the default parameters will be used while the Monte Carlo default parameters do not run a simulated annealing calculation. If the additional options "mc_sim_an" and "mc_sim_an_steps" are passed in the options file, a Monte Carlo simulation will change to a simulated annealing simulation.

"mc_sim_an" will determine the scaling factor for the temperature and should be between 0 and 1, often slow cooling such as 0.99 is desirable. The other parameter, "mc_sim_an_steps", determines the amount of Monte Carlo steps after which the temperature is re-scaled by "mc_sim_an".

4.6 Replica Exchange Monte Carlo

The REMC method also builds upon the aforementioned MC Fortran routine but in this approach multiple MC runs at different temperatures are performed in parallel. After a certain amount of MC simulation steps per temperature, structures are interchanged between the independent simulations. The probability at which an interchange between simulations at different temperatures is accepted is [5]: $\min[1, \exp(\Delta E \cdot (1/kT_1 - 1/kT_2))]$. Such a parallel tempering strategy can help to overcome energy barriers by using higher temperatures but still enables local optimizations in the parallel runs at lower temperatures.

This algorithm also relies on the Monte Carlo code and therefore the same parameters apply as discussed for Monte Carlo. The method is selected via "-s 'random-remc'". However, the "mc_write_steps" option has no effect as a summary after each exchange will be written and no direct Monte Carlo output is shown. Moreover, multiple temperatures have to be provided in the options file for the "mc_kt" parameter, all separated by spaces. In addition to the parallelization over independent simulations as specified by the "-n" flag also parallelization over the temperatures is performed. Therefore, the best performance is obtained for setting the environment variable "OMP_NESTED" to True if parallelization over both, independent runs and temperatures is desired. In addition, the "mc_steps" option now defines the amount of Monte Carlo steps after which an replica exchange is attempted and therefore smaller numbers than for pure Monte Carlo should be applied. Finally, an additional option, "re_repeat", has to be specified in the options file that is passed via the "-y" flag. This option specifies how many times an replica exchange should be performed, each after, "mc_steps" Monte Carlo steps for each temperature "mc_kt". The abortion options "stop_time" (in seconds) and "stop_steps_no_improve" are also available. "tol" (tolerance in energy at which structures are considered to be really different and not just numerical fluctuations) should be specified in the options file for abortion by steps without improvement.

4.7 Genetic Algorithm

A Fortran implementation of a genetic algorithm[6] for the atomistic combinatorial problem which starts from a pool of random structures. Parent structures are selected via roulette wheel method[7] and the structures are crossed by randomly selecting atom-swaps that are different in the parent structures with an expectation value of accepting 50% of all possible atom-swaps that constitute the differences in the parent structures. Moreover, random mutation can be applied to the generated structures at a given rate. Furthermore, elitism is supported by directly transferring a certain amount

of best structures from the parent pool to the child pool of the next generation (self-reproduction). The algorithm can be run for a desired amount of steps (generations).

The Genetic Algorithm can be accessed via selecting the "random-ga" solver with help of the "-s" flag which will cause in using the default parameter set. Again, all parameters can be specified within an options file that is passed with the "-y" flag. The "-n" flag determines how many solutions for the initial generation are created randomly. The rest of the generation will be also filled by random structures so this feature is only helpful if an elaborate guess for the first generation is made internally in the Python code. For use via the command line simply "-n generation_size" or lower should be used as "-n" still controls how many of the lowest energy structures found are stored. In addition to that, the "ga_steps" and "ga_write_step" parameters should be specified in the options file to select the number of generations that are created and to control the verbosity at which generation summaries are printed. Internally, the algorithm will create "pool_size" structures from the "generation_size" parent structures and will select the lowest "generation_size" structures from the pool for the next generation. Both parameters have to be specified in an options file. Within the internal creation of structures also mutation at the rate of "mutation_rate" is applied to the generated structures. When setting this parameter, it should be considered that the given rate is per site position so a rate of 0.01 in a system with 100 iterative sites would result in the expectation of 1 mutation (site swap) for each structure. Therefore, lower values are usually best to ensure that also non-mutated structures will be in the structure pool. The selection of the generation from the pool can be also controlled via the "elite_size" parameter in an options file. The specified number of structures will be directly transferred from the lowest energy parent generation structures to the next generation. This setting can be very useful to ensure that the current global minimum structure is always in the current generation. Lastly, the genetic algorithm also supports abortion by "stop_time" (in seconds) and "stop_steps_no_improve" as discussed for the other solvers as well. "tol" (tolerance in energy at which structures are considered to be really different and not just numerical fluctuations) should be specified in the options file for abortion by steps without improvement.

4.8 Hybrid Approach

As the aforementioned algorithms can get trapped in local minima which prevents them from reaching the global optimum, it can be beneficial to combine different methods to a hybrid heuristic approach. Especially the GA can run into a local minimum and if too little variation is present in the generation, the algorithm gets trapped. In such cases, it can help to perform local minimization of the generation structures or to apply more dynamic optimization approaches to overcome local barriers and thereby improve the variations in the generation structures.[8] Combinations of a GA with MC, SA, and REMC were all proposed and proven to be effective for chemical optimization problems.[9, 10] In GOAC, such hybrid heuristics can be conveniently created by calling the different Fortran heuristic methods sequentially from a Python script. Per default, a hybrid approach build from

Replica Exchange Monte Carlo and Genetic Algorithm can be chosen.

The hybrid approach of Replica Exchange Monte Carlo and the Genetic Algorithm is available by calling "-s 'random-hybrid'". In the options file the parameters for the two pure methods discussed above should be given. It is probably most useful to specify the "stop_time" option as this will apply to Replica Exchange Monte Carlo and the Genetic Algorithm and therefore allows to control when a switch between the two methods is performed. Moreover, one additional parameter, "hybrid_repeat", should be specified in the options file that is passed with the "-y" flag. This parameter controls how often both pure methods are repeated in an alternating matter. The code always starts with a Replica Exchange Monte Carlo run and the best solutions of each independent run (specified by "-n") are used for the initial generation of the Genetic Algorithm and from the lowest energy structures of the last generation in the Genetic Algorithm independent Replica Exchange Monte Carlo runs are started again. It can be worth considering running plenty of Replica Exchange Monte Carlo simulations in parallel for just a short time to build the generations for the Genetic Algorithm (almost) exclusively from Replica Exchange Monte Carlo structures and just a few random structures to give some additional variation.

References

[1] S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson, G. Ceder, *Computational Materials Science* **2013**, *68*, 314–319.

[2] K. Momma, F. Izumi, *Journal of Applied Crystallography* **2008**, *41*, 653–658.

[3] L. L. Gurobi Optimization, Gurobi Optimizer Reference Manual, **2023**.

[4] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, E. Teller, *The Journal of Chemical Physics* **1953**, *21*, 1087–1092.

[5] C. Thachuk, A. Shmygelska, H. H. Hoos, *BMC bioinformatics* **2007**, *8*, 342.

[6] A. S. Fraser, *Australian Journal of Biological Sciences* **1957**, *10*, 484.

[7] A. Lipowski, D. Lipowska, *Physica A: Statistical Mechanics and its Applications* **2012**, *391*, 2193–2196.

[8] S. K. Gregurick, M. H. Alexander, B. Hartke, *The Journal of Chemical Physics* **1996**, *104*, 2684–2691.

[9] N. Dugan, Ş. Erkoç, *Computational Materials Science* **2009**, *45*, 127–132.

[10] Y. Sakae, T. Hiroyasu, M. Miki, K. Ishii, Y. Okamoto, *Molecular Simulation* **2015**, *41*, 1045–1049.